NIRANTAR

GENOVATE 2.0 — Product Requirements Document

Live Railway Intelligence • Passenger Flow • Predictive Operations • Voice AI • Secure Transactions

Version	1.0 — Competition Build
Target event	GENOVATE 2.0 — 26 September 2026
Build window	19 days from 7 September 2026
Primary goal	Turn the existing Nirantar prototype into a credible, live-data-driven competition demo
Scope principle	Add only features that strengthen the railway intelligence story

Executive decision

Do not rebuild Nirantar from scratch. Preserve the existing core product and add high-impact capabilities: live/near-live train telemetry, passenger device geolocation, event-driven operational simulation, predictive alerts, voice interaction, and a sandboxed transaction flow.

Integrity rule: any train feed must be labelled according to its actual provenance. If a third-party feed is used, call it third-party live data. If the feed is simulated, call it simulated telemetry. Never present simulated positions as official Indian Railways GPS.

Document status: Ready for implementation planning

1. Product Vision

Nirantar is an intelligent railway passenger-and-operations layer that combines live transport telemetry, passenger location, passenger-flow intelligence, prediction and conversational assistance to help users understand what is happening now, what is likely to happen next, and what action should be taken.

Competition positioning

The strongest story is not “a railway dashboard.” It is an event-driven operational intelligence system: changes in train movement, delay, passenger density or platform conditions propagate through the system and produce updated forecasts, alerts and recommendations.

Core demo promise

“Nirantar does not only report railway conditions. It continuously interprets changing conditions and recommends what passengers or operators should do next.”

Success criteria

Goal	Target
Judge comprehension	Problem and value understood within 45–60 seconds
Live feel	At least one genuine live data source + device location
Operational reaction	One simulated delay/incident visibly changes downstream state
AI relevance	AI answers questions and/or explains recommendations using system state
Reliability	Critical demo path works without manual database edits
Integrity	External and simulated data clearly distinguished

2. Scope & Priorities

P0 features are the minimum competition build. P1 features are valuable if the core path is stable. P2 features should be cut first if time becomes constrained.

Priority	Feature	Reason
P0	Live/near-live train running status	Transforms the map from a static visual into a data product
P0	Train route + coordinates + delay + ETA	Creates concrete operational state
P0	Browser passenger geolocation	Demonstrates real device telemetry
P0	Live event/state engine	Lets one event update the whole system
P0	Predictive congestion/alert layer	Makes AI/ML operational rather than decorative
P0	Voice query/action layer	Shows accessible conversational interaction
P1	Razorpay Test Mode transaction	Demonstrates secure event-driven payment without real money
P1	Passenger service/wallet ledger	Connects transaction capability to railway use case
P1	Historical trend charts	Adds analytical depth
P2	Real bank payout / real money	Not required; adds security/compliance risk
P2	Full ticket booking	Too large for the remaining build window

3. Target System Architecture

Frontend (existing Nirantar) → API layer → data-provider adapters → normalized telemetry → state engine → prediction/rules → UI + voice. Keep provider-specific payloads out of the UI.

Frontend
• Next.js / React (existing)
• Mapbox GL JS or Google Maps JavaScript API
• Live state via SSE/WebSocket or short polling
• Browser Geolocation API

Backend
• Next.js API routes / Node service
• Provider adapters + normalization
• Event/state engine
• Prediction service / existing ML models
• Razorpay server integration + webhook handler

Storage
• Postgres/Supabase or existing DB
• Redis optional for short-lived live state/cache
• Append-only event log for observability

Normalized telemetry object

{ trainId, trainNumber, timestamp, latitude, longitude, speedKmh, currentStation, nextStation, scheduledArrival, estimatedArrival, delayMinutes, platform, source, sourceFreshnessSeconds }

The source and sourceFreshnessSeconds fields let the UI honestly show whether information is live, cached, stale or simulated.

4. External Data & API Strategy

Use provider adapters so Nirantar is not locked to one railway API. Validate access, freshness, quotas, licensing and reliability before depending on any provider.

Need	Preferred option	Backup	Notes
Live train running status	RailRadar API	RailKit / APIclub / Indian Rail API	Validate access, freshness, quota and terms
Train route/stations	Same provider	Static route dataset	Cache stable route metadata
Map	Mapbox GL JS	Google Maps JavaScript API	Choose one
Passenger GPS	Browser Geolocation API	Manual demo coordinates	Requires user permission + HTTPS
Payment	Razorpay Test Mode	Internal mock	No real money required
AI/voice	Existing Nirantar integrations	Text-only fallback	Voice never blocks core demo

Current railway-data candidates

RailRadar documents a live-train endpoint with delay, current telemetry, next halt, platform updates and route geometry; its site advertises a developer API and a free sandbox tier with 1,000 requests/month. Treat it as a third-party provider, not an official Indian Railways API. □cite□turn1search0□turn1search8□

Indian Rail API documents live-train status with current station, scheduled/actual arrival/departure and delay information. Validate current reliability before use. □cite□turn1search1□

RailKit advertises live train tracking, PNR, seat availability and station intelligence through a developer API/SDK; it is another candidate for comparison. □cite□turn1search6□

Do not scrape protected pages or bypass access controls. Confirm provider terms before using live data publicly.

5. Live Map & GPS Requirements

Mapbox GL JS supports custom sources/layers and a GeolocateControl using browser geolocation; Mapbox also documents updating a map from a real-time GeoJSON feed. □cite□turn0search2□turn0search15□

Google Maps JavaScript API supports interactive maps, markers and custom data layers. Google documents browser geolocation using the W3C Geolocation API with user consent and secure-context requirements. □cite□turn0search1□turn0search7□turn0search5□

Recommended implementation

Use one mapping provider. Mapbox is strong for a custom visual experience; Google Maps is strong if the existing stack already uses Google services.

Map behaviour	Requirement
Train marker	Animate from normalized latitude/longitude updates
Route	Render route geometry/polyline
Stations	Clickable markers with schedule + live status
Passenger	Location dot + accuracy radius
Delay	Status chip + timeline indicator
Freshness	Show “Updated X sec/min ago”
Failure	Show last known position + stale warning

GPS flow

HTTPS → user clicks “Locate me” → browser permission → navigator.geolocation/watchPosition → normalized passenger position → nearest station calculation → map update. Do not request location silently. □cite□turn0search5□turn0search6□

6. Real-Time Train Data Pipeline

Polling is sufficient for the competition build; you do not need a complex streaming platform.

```
Provider API → Provider Adapter → Schema Normalizer → Validation + freshness check → Cache → Event/state engine → SSE/WebSocket or 5–15 s client polling → Nirantar UI
```

Recommended refresh

- Provider polling: 15–60 seconds depending on quota/freshness.
- UI updates immediately when normalized state changes.
- Mark data stale after a configurable threshold such as 2–3× the expected interval.
- Never call an upstream provider separately from every browser; centralize calls server-side and cache.

Derived fields

- Delay = actual/estimated timing − scheduled timing.
- ETA = current time + estimated remaining travel time.
- Distance-to-next-station = route/geodesic distance.
- Risk score = delay + passenger density + arrival proximity + platform capacity.
- Freshness = current time − provider timestamp.

7. Event & Simulation Engine

The judge should be able to trigger an event and see Nirantar react immediately.

Event	Input change	Expected reaction
Train delay +5/+15 min	delayMinutes changes	ETA, board, advice and risk recalculate
Platform change	platform changes	Passenger guidance + alerts update
High passenger density	density rises	Congestion risk rises; recommendation generated
Train approaching	ETA threshold crossed	Arrival alert + crowd forecast
Cancellation	status = cancelled	Affected journey state + alternatives
GPS update	passenger coordinates change	Nearest station/platform context updates

Demo control panel

[TRAIN 12841] [Normal] [+5 min] [+15 min] [Platform Change] [High Density] [Cancellation] [Reset Scenario]

Label this as a Demo / Simulation Console. It is a test harness that proves the system responds to changing operational conditions.

8. AI/ML & Predictive Layer

AI must be connected to operational decisions, not added as a chatbot ornament.

Recommended outputs

- Passenger-flow forecast for 5/10/15 minutes.
- Platform congestion risk: Low / Medium / High / Critical.
- Train arrival impact on crowd density.
- Suggested passenger redistribution or alternate-platform guidance.
- Natural-language explanation of why a recommendation was generated.

Example: Train arriving in 7 minutes + platform density 82% + historical arrival surge → “High congestion risk in ~8 minutes. Consider redirecting incoming passengers toward Platform 2.”

Model contract

POST /api/predict { stationId, platformId, currentDensity, arrivingTrains, delayMinutes, horizonMinutes } → { congestionProbability, riskLevel, predictedDensity, recommendedAction, explanation }

Reuse existing Nirantar ML models if they work. Only retrain if a clear performance or demo requirement exists.

9. Voice AI Layer

Use the existing Deepgram + Murf integration as a thin conversational layer over Nirantar's normalized state.

User asks	System action
“Where is my train?”	Fetch selected train state and speak location/ETA
“Is platform 3 crowded?”	Read current density + risk
“What should I do?”	Return recommendation
“Why is there an alert?”	Explain from structured state

The LLM must not invent operational facts. Give it structured Nirantar state and constrain answers to available fields. If the LLM fails, fall back to deterministic text responses.

10. Razorpay Transaction Module

Do not use real bank credentials or real money. Use Razorpay Test Mode. Razorpay documents Test Mode as a sandbox where actions do not affect the live environment; its webhook documentation supports testing event delivery and signature validation. cite turn0search10 turn0search3

Railway-specific use case

Implement a small “Passenger Services Wallet / Assistance” flow. Example: passenger pays a ₹50 virtual service fee in Test Mode; the webhook marks the service as paid; Nirantar updates a transaction ledger. This gives the payment feature a railway purpose.

Architecture

Passenger → Create Order → Razorpay Checkout (TEST) → Payment success → Webhook → Signature verification → Idempotency check → Transaction ledger → Service = PAID

Endpoint	Purpose
POST /api/payments/order	Create server-side test order
POST /api/payments/verify	Verify payment result
POST /api/webhooks/razorpay	Receive and validate webhook
GET /api/payments/:id	Return transaction status

Security

- Never expose secret keys in the browser.
- Verify webhook signatures server-side.
- Make webhook processing idempotent.
- Store minimum transaction metadata.
- Use Test Mode throughout development and rehearsals.

11. Minimal Data Model

Entity	Key fields
Train	id, number, name, source, destination, routeId
TrainTelemetry	trainId, timestamp, lat, lng, speed, bearing, delay, currentStation, nextStation, platform, source
Station	id, code, name, lat, lng
Platform	id, stationId, number, capacity
PassengerSession	id, lat, lng, accuracy, selectedTrainId, selectedStationId
Prediction	timestamp, stationId, platformId, predictedDensity, risk, recommendation
Event	id, type, timestamp, payload, scenarioId
Payment	id, orderId, paymentId, amount, status, serviceType, createdAt

Do not overbuild the database. It exists to make state persistent and traceable, not to become the project itself.

12. Nirantar Internal API Contract

Method	Endpoint	Purpose
GET	/api/trains/:number/live	Normalized live train state
GET	/api/trains/:number/route	Route + geometry
GET	/api/stations/:code	Station metadata
GET	/api/stations/:code/board	Arrivals/departures
GET	/api/passenger/location	Current session location
POST	/api/predict	Prediction/risk
POST	/api/events	Trigger simulation event
GET	/api/events/state	Current scenario state
POST	/api/payments/order	Create test order
POST	/api/payments/verify	Verify test payment
POST	/api/webhooks/razorpay	Payment events
POST	/api/voice/query	Voice/text → structured answer

All provider calls should be server-side. The browser should never receive private railway API credentials or Razorpay secrets.

13. Reliability, Security & Failure Modes

Failure	Fallback
Railway API unavailable	Last known state + stale indicator; switch to simulation
API quota exhausted	Cached route data + demo telemetry
Map unavailable	Simplified route/timeline view
GPS denied	Manual station selection / demo coordinates
Voice unavailable	Text input/output
Razorpay unavailable	Clearly labelled demo transaction simulator
ML unavailable	Rule-based risk fallback

Security checklist

- HTTPS deployment.
- Environment variables for private keys.
- Server-side provider proxy.
- Rate limiting on public endpoints.
- Input validation.
- Webhook signature validation + idempotency.
- No real bank credentials or production financial accounts.
- Visible provenance: LIVE / CACHED / SIMULATED.

14. Testing Plan

Test	Acceptance criterion
Train API	Valid train returns normalized telemetry within target latency
Freshness	Stale data is labelled
Map	Marker follows returned coordinates and route
GPS	Permission works; denial has fallback
Delay event	ETA/alerts update
Density event	Risk/recommendation update
Prediction	Known test cases produce expected risk bands
Voice	Questions map to correct state
Payment	Test payment completes without real money
Webhook	Signature validates; duplicates do not duplicate ledger
Failure	Provider outage does not crash demo

Load testing is secondary. First prove correctness, freshness, recovery and deterministic demo behaviour.

15. 19-Day Execution Plan

Days	Deliverable	Definition of done
1–2	Architecture + provider tests	Railway provider and map choice confirmed
3–5	Live train pipeline	Train state normalized and displayed
6–7	Map + GPS	Train + passenger location work on HTTPS
8–9	Event engine	Delay/platform/density events update UI
10–11	Prediction	Risk + recommendation integrated
12	Voice	3 useful railway questions work
13–14	Razorpay Test Mode	Order → payment → webhook → ledger
15	Fallback states	Demo survives provider/API failures
16	Visual polish	Hierarchy, spacing and motion cleaned up
17	Demo script	3-minute and 5-minute versions rehearsed
18	Dress rehearsal	Fresh deployment tested end-to-end
19	Freeze	No new features; bug fixes only

After Day 17, no new feature should be added unless it fixes a judge-facing failure.

16. Competition Demo Script

Time	Action	Judge takeaway
0:00–0:20	Open map + selected train	Live railway intelligence interface
0:20–0:45	Show location, speed, ETA, delay, next station	Operational telemetry
0:45–1:05	Tap Locate Me	Passenger context is live
1:05–1:30	Trigger +15 min delay	System is event-driven
1:30–1:55	Show forecast/risk/recommendation	AI converts state into action
1:55–2:15	Ask voice question	Conversational access
2:15–2:40	Run Test Mode service payment	Secure transaction flow
2:40–3:00	Explain architecture + differentiation	Technical depth + applicability

Closing line

“Nirantar connects live railway conditions, passenger context and predictive intelligence into one continuous operational loop — helping users understand not only what is happening, but what is likely to happen next and how to respond.”

17. What NOT to Build

- Real-money bank transfers.
- Full IRCTC ticket booking.
- A new mobile app unless browser UX is complete.
- A new ML model solely for presentation.
- Multiple map providers at once.
- A huge railway-event database.
- A general-purpose chatbot disconnected from Nirantar state.
- Unverified claims of official Indian Railways data.
- Features that cannot be demonstrated reliably in under 3 minutes.

Founder-level decision rule

Every feature must answer: Does it improve live realism? prediction/actionability? judge comprehension? technical depth? If not, cut it.

18. External References & API Candidates

These references were checked while preparing this PRD. Availability, pricing, quotas, terms and endpoint behaviour can change; verify immediately before integration.

Resource	Use
RailRadar Live Train API	Candidate live train status, telemetry, delay and route geometry
Indian Rail API	Candidate live train status and route/station data
RailKit	Candidate live train tracking / railway API
Mapbox GL JS	Interactive map, custom layers, browser geolocation
Google Maps JavaScript API	Interactive map, markers, geolocation
Razorpay Test Mode	Sandbox payments
Razorpay Webhooks	Webhook validation, signature and testing

Reference URLs

RailRadar: https://railradar.in/docs/live-train-status
Indian Rail API: https://indianrailapi.com/api-collection/live-train-status
RailKit: https://railkit.in/
Mapbox live data: https://docs.mapbox.com/mapbox-gl-js/example/live-geojson/
Google Maps geolocation: https://developers.google.com/maps/documentation/javascript/geolocation
Razorpay Test Mode: https://razorpay.com/docs/x/dashboard/test-mode/
Razorpay Webhooks: https://razorpay.com/docs/webhooks/validate-test/

Final implementation priority

Live train telemetry → live map → passenger GPS → event simulation → prediction/recommendation → voice. Razorpay is a supporting module, not the headline feature.